

Persistencia de objetos con JSONB

1. Persistencia de objetos con JSONB

Persistencia de objetos con JSONB

Hasta ahora, cada atributo de tus entidades se almacenaba normalmente en una columna independiente. Este apartado introduce una posibilidad intermedia entre el modelo relacional clásico y las bases de datos orientadas a objetos: utilizar capacidades objeto-relacionales de PostgreSQL para almacenar datos estructurados dentro de una columna.

Con jsonb, PostgreSQL no guarda directamente un objeto Java ni sus métodos. Guarda su representación como un documento JSON, que Hibernate puede convertir al escribir y reconstruir al leer.

Repaso rápido: qué es JSON

JSON (JavaScript Object Notation) es un formato de texto para representar datos estructurados: objetos (pares clave-valor entre {}), arrays (listas entre []), y valores anidables entre ambos:

```
{
  "tapaDura":{
    "isbn":"978-84-376-0494-7",
    "paginas":496
  },
  "ebook":{
    "formato":"epub"
  }
}
```

Se ha convertido en el formato universal de intercambio de datos porque es legible por humanos, ligero, y prácticamente todos los lenguajes lo leen y escriben de forma nativa.

Objeto simple vs. objeto estructurado

- Objeto simple: un valor escalar — un texto, un número, una fecha. Es lo que has persistido siempre (título, precio).
- Objeto estructurado: un objeto con campos anidados, posiblemente con colecciones dentro. En el ejemplo anterior, se almacenan los datos de dos ediciones del mismo libro —tapaDura y ebook— y cada una tiene sus propios atributos.

Qué ofrece PostgreSQL: el tipo jsonb

PostgreSQL tiene un tipo de columna especial, jsonb (JSON binario), que permite que una fila normal, de una tabla normal, lleve dentro un objeto JSON completo —con estructura que puede variar de una fila a otra— conviviendo con las columnas y claves foráneas de siempre.

No hace falta una base de datos documental aparte (eso lo verás en el Tema 3), ni tampoco una conexión distinta: sigue siendo el mismo PostgreSQL, el mismo DataSource del Tema 1, gestionado exactamente igual.

json vs. jsonb, en una frase

PostgreSQL tiene dos tipos: json guarda el texto tal cual, sin analizarlo; jsonb lo analiza y lo guarda en un formato binario indexable y más rápido de consultar — por eso jsonb es casi siempre la opción preferida, salvo que necesites preservar el texto original exacto (espacios, orden de claves).

Un ejemplo completo: detallesEdicion

Cada libro puede existir en varios formatos (tapa dura, bolsillo, ebook), cada uno con sus propios datos internos — un objeto estructurado dentro de la entidad Libro:

```
@Entity
@Table(name = "libro")
public class Libro {

    // ... id, titulo, precio, fechaPublicacion ...

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(columnDefinition = "jsonb")
    private Map<String, Object> detallesEdicion;
}
```

Qué hace cada anotación

`@Column(columnDefinition = "jsonb")` le dice a Hibernate qué tipo de columna PostgreSQL usar; `@JdbcTypeCode(SqlTypes.JSON)` le dice cómo tratar ese campo — como JSON, no como un texto plano cualquiera.

El tipo Java es un simple `Map<String, Object>`: no necesitas una clase Java específica para cada estructura posible, el mapa acepta cualquier forma de JSON.

titulo/precio vs. detallesEdicion

titulo y precio son la persistencia "de toda la vida" del Tema 1 — un valor escalar por columna. detallesEdicion, en cambio, representa datos estructurados que Hibernate serializa como JSON dentro de una única columna:

```
{
  "tapaDura":{ "isbn":"978-84-376-0494-7", "paginas":496 },
  "ebook":{ "formato":"epub" }
}
```

Cómo Hibernate serializa el Map a JSON

Hibernate necesita una biblioteca capaz de convertir el `Map<String, Object>` a JSON y reconstruirlo después. En este proyecto puede utilizar Jackson, que ya está disponible porque Spring Boot lo incorpora mediante las dependencias web que añadiste anteriormente.

Al detectar `@JdbcTypeCode(SqlTypes.JSON)` y encontrar Jackson en el classpath, Hibernate lo utiliza para esa conversión. Por eso no necesitas añadir una dependencia ni escribir un conversor manual.

Por qué JSONB y no una tabla nueva

Con lo que ya sabes del Tema 1, podrías haber modelado esto como una tabla detalle_edicion con @OneToMany. ¿Por qué elegir JSONB?

	Modelo relacional	Columna JSONB
Añadir otra edición igual	Insertar otra fila	Modificar el documento JSON
Añadir atributos nuevos	Puede requerir columnas/tablas nuevas	Normalmente no requiere cambiar el esquema
Integridad y validación	El motor aplica tipos y claves foráneas	La estructura interna no se valida por defecto
Acceso desde Java	Entidades y atributos tipados	Con Map, claves y tipos se comprueban en ejecución

La pérdida de tipado, en la práctica

Con una tabla relacional, escribir `detalleEdicion.getPaginas()` que no existe da un error de compilación. Con `Map<String, Object>`, en cambio, `detallesEdicion.get("paginas")` compila sin problema aunque hayas escrito "pagina" por error: el fallo no aparece hasta que ejecutas el código y te encuentras con un null inesperado.

El compilador ya no te cubre las espaldas — pasa a ser responsabilidad tuya, en tiempo de ejecución.

Qué es un índice

Un índice es una estructura auxiliar que el motor de base de datos mantiene aparte de la tabla, pensada para encontrar filas rápido sin recorrerlas todas — la misma idea que el índice de un libro: en vez de leer página por página, vas directo a la página que el índice señala.

Sin un índice adecuado para la consulta, PostgreSQL puede tener que recorrer la tabla entera, fila a fila.

B-tree y GIN: dos tipos de índice distintos

El tipo más habitual es el B-tree, apropiado para igualdad, comparaciones por rango y ordenación sobre valores escalares: buscar un título concreto u ordenar por precio.

Pero las consultas habituales sobre JSONB no comparan el documento completo: preguntan si existe una clave, si contiene cierta estructura, o si un valor aparece dentro. PostgreSQL resuelve esto con índices GIN (Generalized Inverted Index), diseñados para localizar elementos dentro de valores compuestos como JSONB, arrays o texto.

Crear el índice GIN y usarlo

```
CREATE INDEX idx_libro_detalle_edicion  
ON libro USING GIN (detalle_edicion);
```

El operador ? permite comprobar si existe una clave en el nivel superior del documento:

```
SELECT * FROM libro  
WHERE detalle_edicion ? 'ebook';
```

El índice GIN puede acelerar la consulta. Eso no significa que PostgreSQL vaya a utilizarlo siempre: el planificador compara costes y puede preferir recorrer la tabla completa con pocas filas.

Para que el índice resulte útil

Suelen coincidir dos condiciones: que exista un volumen considerable de datos, y que la consulta sea selectiva (descarte la mayoría de las filas).

Con una tabla de prueba de un puñado de filas no verías ninguna diferencia real: el motor decide el plan por coste estimado, y con pocos datos un recorrido completo ya es barato de por sí. En la Actividad 2.1 vas a generar 100.000 filas y comparar el plan de la misma consulta antes y después de crear el índice.

NULL de la columna vs. null dentro del JSON

Caso	Qué significa
La columna <code>detalles_edicion</code> es NULL	El libro no tiene ningún dato de edición guardado — ni siquiera un objeto vacío.
<code>{"ebook": null}</code>	Hay un objeto JSON guardado, con la clave "ebook" presente, pero su valor es el null de JSON.
<code>{}</code> (objeto vacío)	Hay un objeto JSON guardado, sin ninguna clave dentro.

Comprobar «¿existe la clave ebook?» da resultados distintos en cada caso: NULL de columna da NULL; en `{"ebook": null}` la clave existe; en `{}` no existe.

Ideas clave

- PostgreSQL es un gestor objeto-relacional: mantiene el modelo de tablas y relaciones, pero incorpora tipos avanzados como JSONB para datos estructurados.
- JSONB guarda JSON en formato binario indexable — preferible a JSON (texto plano) salvo que necesites preservar el formato exacto.
- `@JdbcTypeCode(SqlTypes.JSON)` + `@Column(columnDefinition = "jsonb")` mapean un `Map<String, Object>` Java contra una columna jsonb.
- Objeto simple = valor escalar; objeto estructurado = objeto anidado con posibles colecciones dentro.

Ideas clave (cont.)

- JSONB gana en flexibilidad de esquema, pierde integridad referencial y tipado estricto frente a una tabla relacional — una decisión de diseño con trade-offs, no una opción siempre superior.
- JSONB no abre ninguna conexión nueva: reutiliza el mismo DataSource del Tema 1.
- Un índice GIN acelera consultas por el contenido de una columna JSONB — un B-tree no sabe mirar dentro del objeto. Solo gana si la consulta es selectiva además de haber volumen.
- NULL de columna, null dentro del JSON, y objeto vacío {} son tres estados distintos, con un resultado diferente al comprobar "existe la clave".